



CashMonkey

App per la gestione delle finanze personali



01

Analisi dei requisiti



Funzionalità e requisiti

Tracciamento dei movimenti

Registrazione di entrate e uscite, suddivise per categoria (salute, svago, trasporti, ecc.) e con relativo metodo di pagamento, confluenti nei risparmi, per offrire una visione chiara e strutturata dei movimenti dell'utente..

Obiettivo economico

Definizione di un obiettivo economico, comprensivo di importo e scadenza, finalizzato a una gestione consapevole delle proprie risorse per il suo raggiungimento.

Promemoria di pagamento

Possibilità di utilizzo di promemoria di pagamento, garantendo maggior consapevolezza sulle scadenze di pagamento. Aiutano l'utilizzatore dell'applicativo fornendo l'importo da pagare, la data e l'ora della scadenza e una descrizione riepilogativa.

Resoconti finanziari

Generazione di resoconti finanziari mensili e annuali contenenti gli estratti conto dei periodi relativi e un'analisi dell'andamento rispetto ai trascorsi precedenti, utili al monitoraggio e alla valutazione della gestione finanziaria.





Funzionalità e requisiti

Monitoraggio risparmi

Monitoraggio dei risparmi attuali calcolati in base al saldo iniziale, alle entrate e alle uscite registrate.

Sistema client-server

Architettura client-server con la presenza di un database per la memorizzazione e la gestione dei dati.

Differenti metodi di pagamento

Personalizzazione completa riguardante i metodi di pagamento con cui l'utente effettua transazioni

Sistema di autenticazione

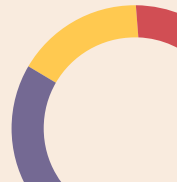
Accesso tramite autenticazione, così che l'utilizzatore possa ritrovare i propri dati da qualunque dispositivo.

Scelta della valuta

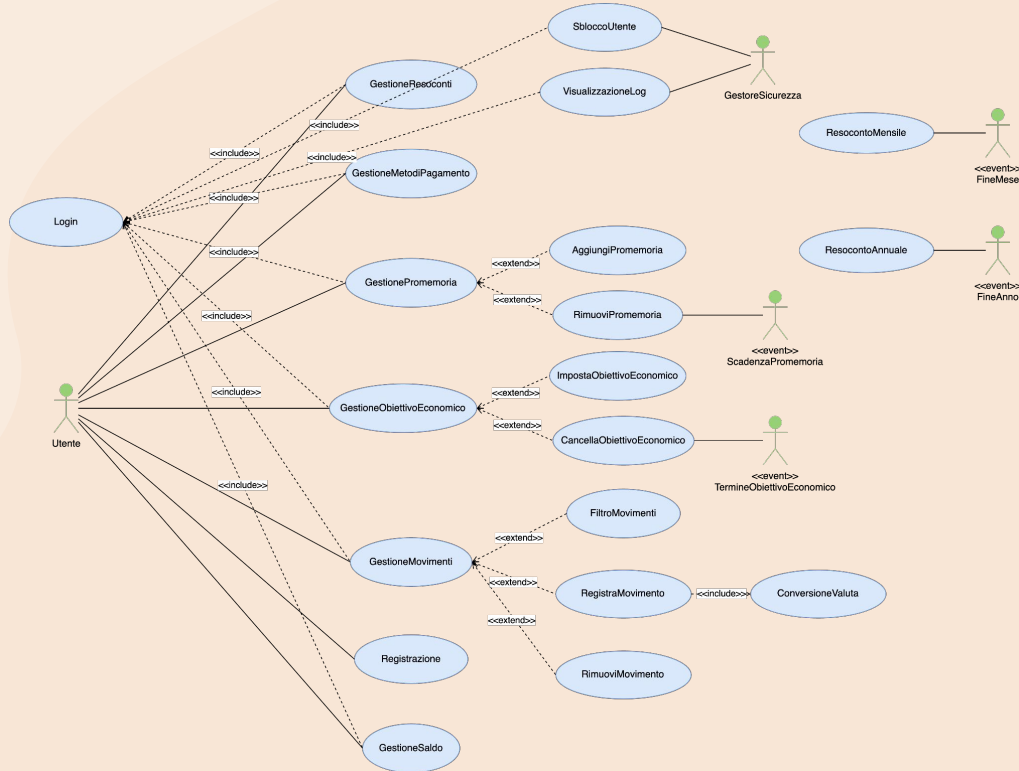
Scelta di una valuta principale su cui si basa l'esperienza di utilizzo dell'applicativo, con annessa conversione automatica degli importi in valute differenti

Ricerca con filtri

Visualizzazione filtrata dei movimenti in base a data, categoria, metodo di pagamento e valuta..



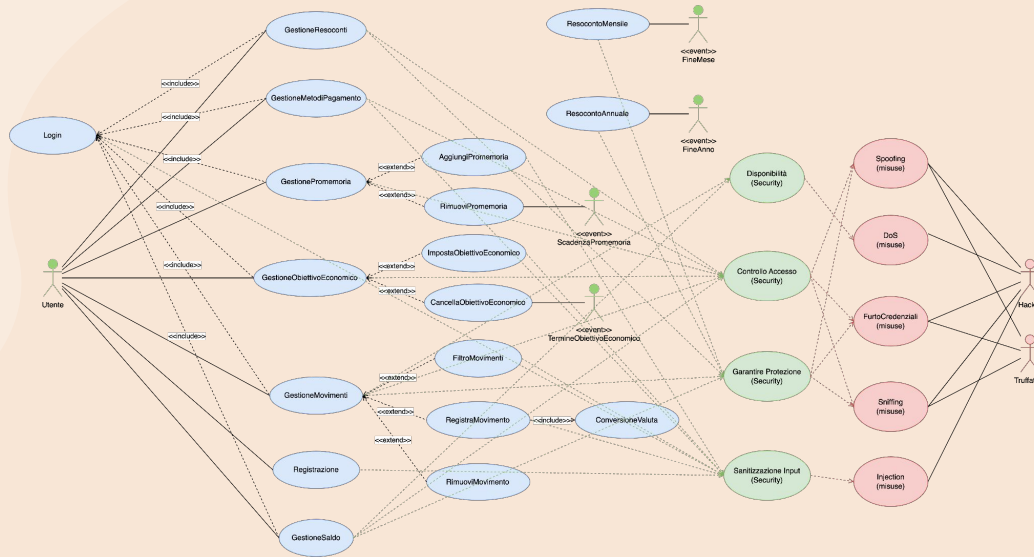
Casi d'uso



Attori

Gli attori che si vedono coinvolti nelle normali operazioni all'interno del sistema sono l'utente utilizzatore (registrazione di movimenti, impostazione dell'obiettivo economico, ecc.) e gli eventi costituiti dalla scadenza dei promemoria o dal giungimento al termine dell'obiettivo economico

Analisi del rischio



Minacce

Il sistema si propone di affrontare e garantire una certa sicurezza rispetto alle seguenti avversità:

- Spoofing
- DoS
- Furto di credenziali
- Sniffing
- Injection



02

Analisi del problema



Modello del dominio

Il modello del dominio, così come l'intero applicativo, è centrato sull'utente, a cui sono legate pressoché tutte le entità e quindi le relative funzionalità.

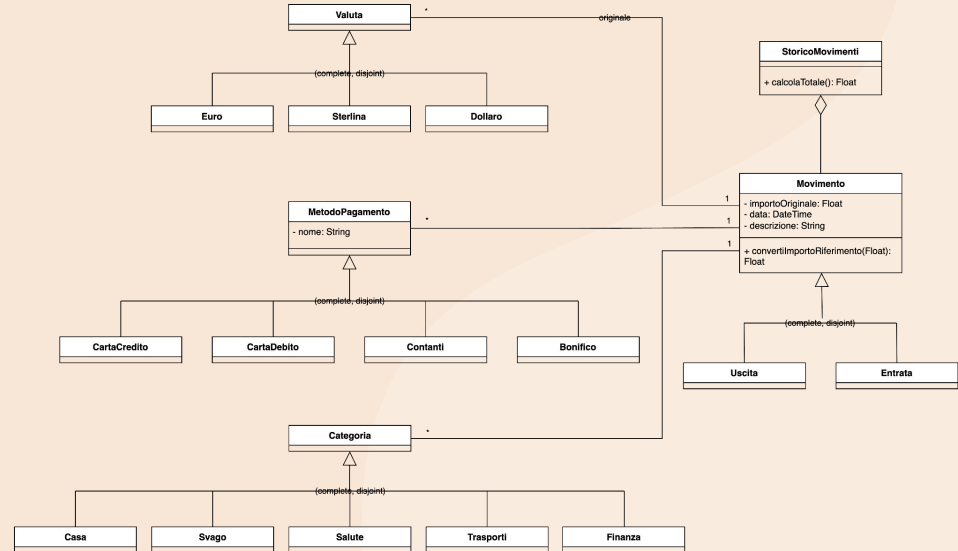
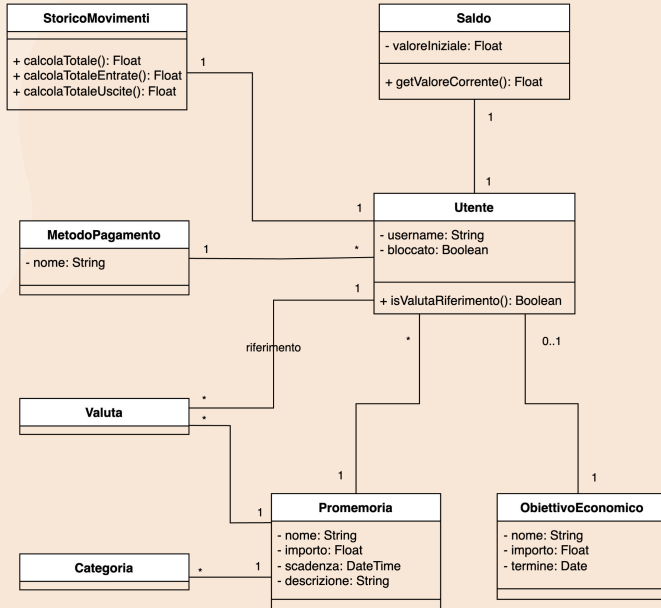
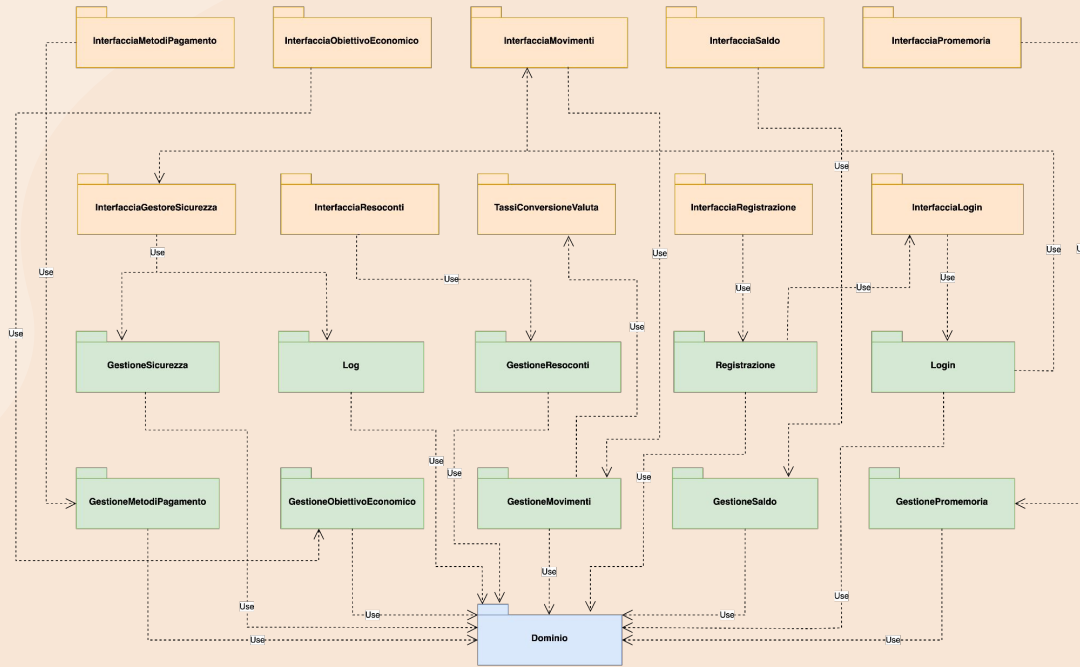


Diagramma dei package

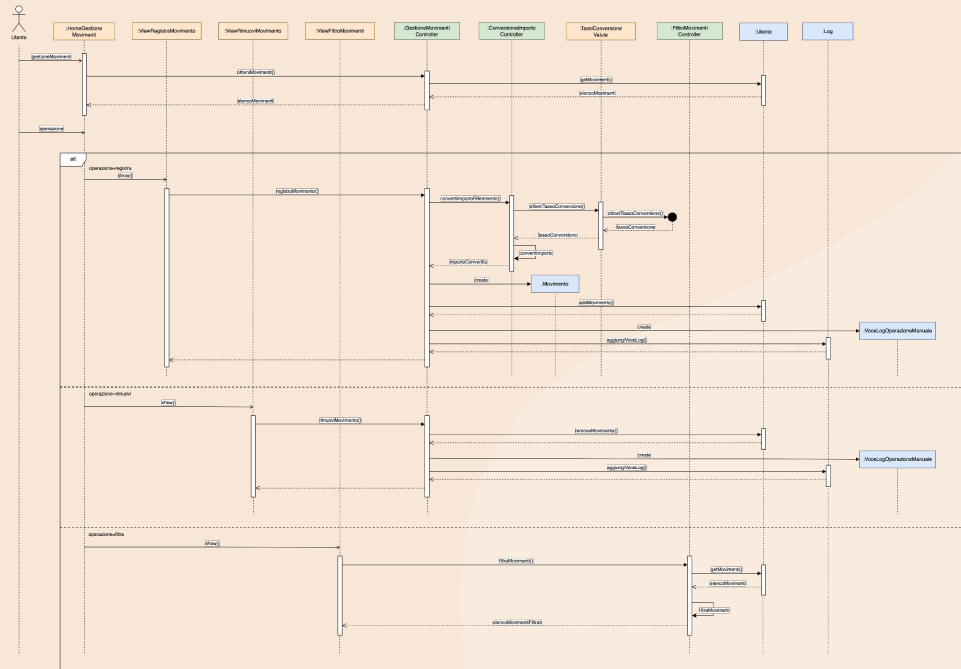
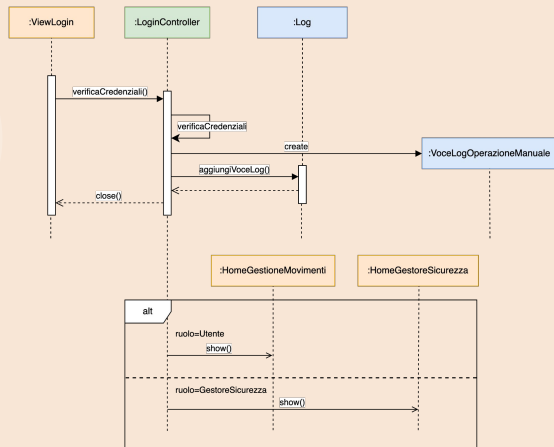


Suddivisione funzionale

L'insieme dei package rispecchia fedelmente le funzionalità offerte dal sistema. In questo modo le varie componenti vengono separate in modo chiaro così da garantire pulizia del codice, maggiore leggibilità e facilità nell'implementazione di eventuali sviluppi futuri.

Diagrammi di sequenza

I diagrammi di sequenza spiegano come gli attori del sistema si interfacciano con le entità emerse e le funzionalità da queste offerte. Tra i più significativi ci sono quelli relativi a login (utente e gestore della sicurezza) e gestione dei movimenti (utente).



Piano del collaudo

Classi di test

È stato definito un abbozzo del piano del collaudo già in questa fase del progetto, per mettere subito per iscritto dei vincoli che ci si aspetta che il sistema soddisfi. Come linguaggio in cui sono stati realizzati i test si è scelto NUnit.

```
[TestFixture]
public class TestUtente
{
    private Utente _utente;
    private string _username;
    private float _saldoIniziale;
    private Valuta _valutaRiferimento;
    private bool _bloccato;

    [SetUp]
    public void UtenteSetUp() {
        _username = "mariorossi";
        _saldoIniziale = 24560.00f;
        _valutaRiferimento = new Euro();
        _bloccato = false;
        _utente = new Utente(_username, _saldoIniziale,
            _valutaRiferimento, _bloccato);
    }

    [Test]
    public void TestGetMethod() {
        Assert.That(_utente.getUsername(), Is.EqualTo(_username));
        Assert.That(_utente.getSaldoIniziale(),
            Is.EqualTo(_saldoIniziale));
        Assert.That(_utente.getValutaRiferimento(),
            Is.EqualTo(_valutaRiferimento));
        Assert.That(_utente.isBloccato(), Is.EqualTo(_bloccato));
    }
}
```

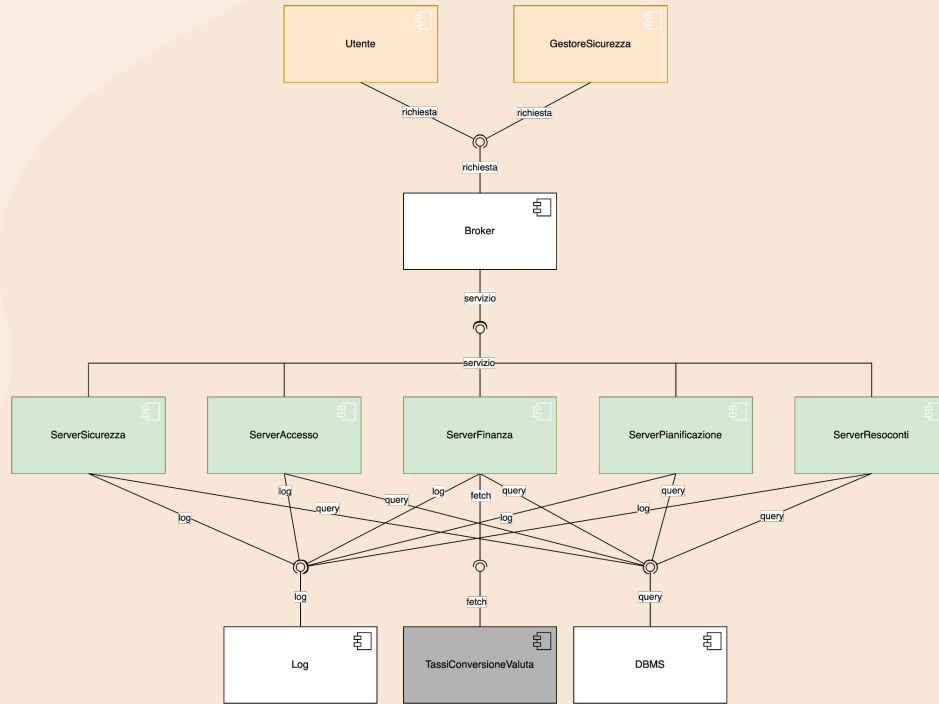


03

Progettazione



Diagramma dei componenti



Scelte architetturali

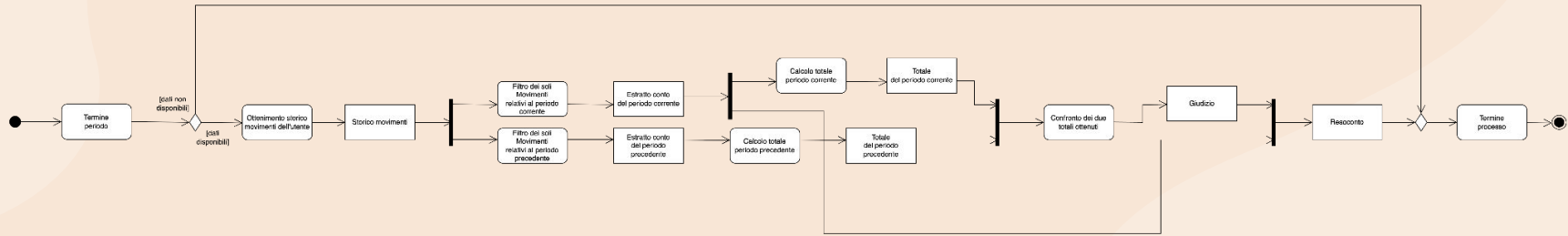
Si è deciso di utilizzare il pattern Broker per assicurare un livello di indirettezza nella comunicazione tra client e server.

Inoltre il sistema è stato distribuito in cinque server, scelti in base al settore delle operazioni che può eseguire l'utente.

Si noti infine la persistenza costituita da un DBMS e dal file dei log, e l'interfacciamento del server relativo alla finanza col sistema esterno per ottenere i tassi di conversione in tempo reale.

Diagramma delle attività

Il diagramma delle attività seguente è valido per la generazione dei resoconti sia mensili che annuali. Il suo scopo è dettagliare in modo esaustivo l'algoritmo per la creazione di tali report, consistenti in: estratto conto del periodo precedente, estratto conto del periodo attuale, giudizio.





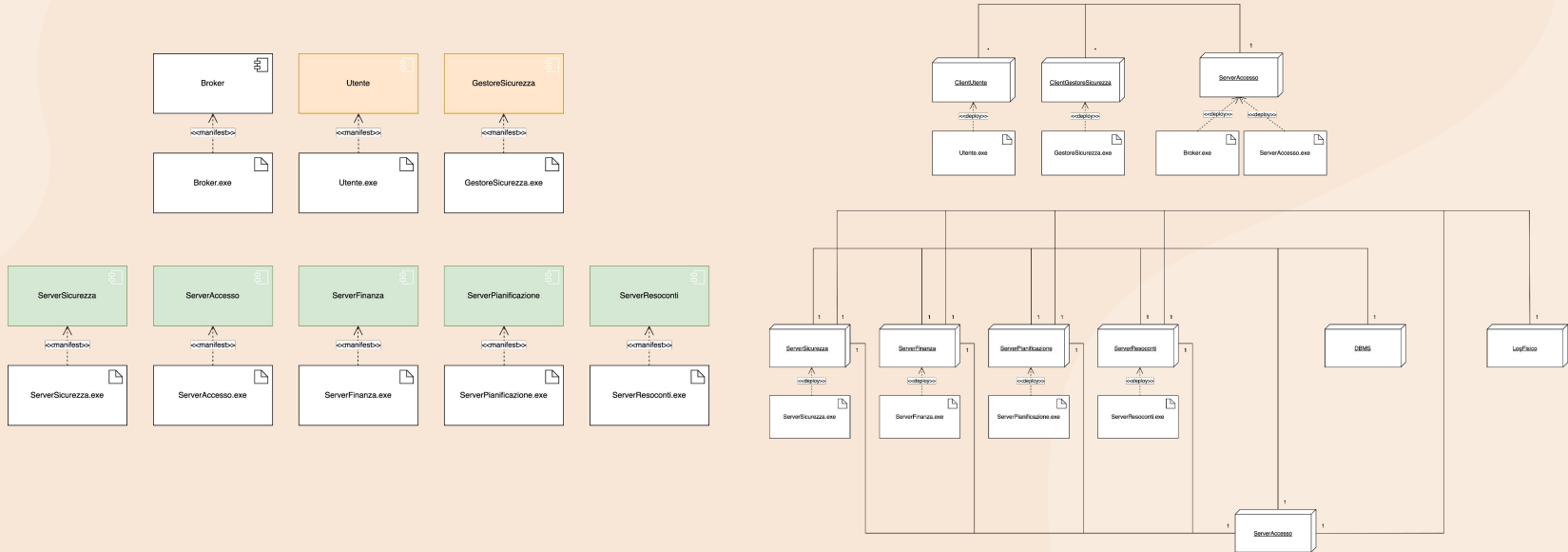
La gestione dei dati applicativi nel sistema è a carico di un DBMS (ad eccezione dei dati della sicurezza relativi ai log che si trovano su file).

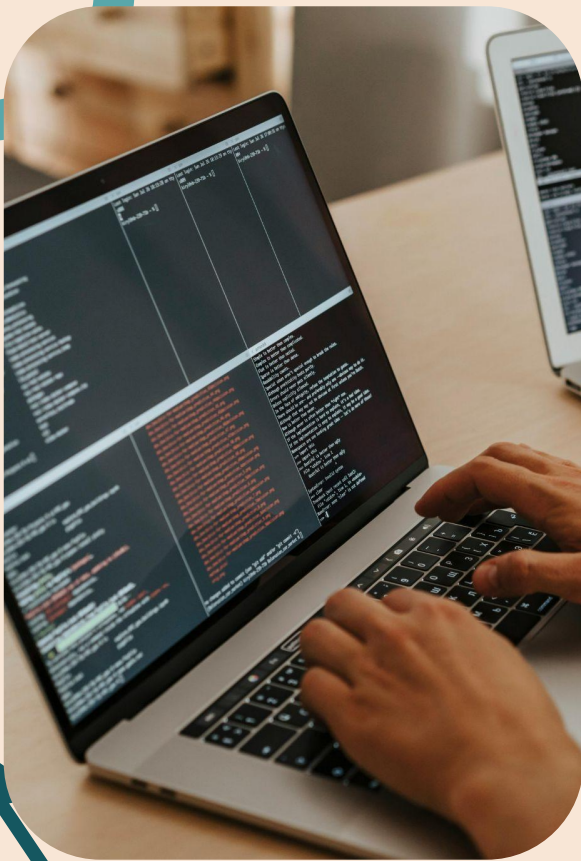
Lo schema E/R è incentrato sull'utente, che fa da PK in molte delle associazioni a cui partecipa.

Progettazione del deployment

I diagrammi del deployment permettono di rappresentare l'architettura fisica del sistema, specificando le relazioni tra componenti software e artefatti utilizzati.

Al centro del diagramma troviamo ServerAccesso connesso a tutti gli altri server specializzati sulle singole funzionalità del sistema. A loro volta, anche i nodi client che eseguono il front-end si collegano ServerAccesso per accedere ai servizi.





04

Implementazione

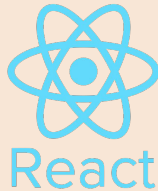


Linguaggi di programmazione

Per realizzare il back-end si è deciso di utilizzare C#, in particolare il framework ASP.NET Core, per svariati motivi, tra cui la possibilità di fare pratica con un nuovo linguaggio di programmazione e la sua integrazione garantita con le tecnologie usate nel front-end.

Nel front-end la scelta è invece ricaduta su React, integrato con Tailwind e Vite. Questi strumenti sono di forte attualità e permettono di costruire interfacce grafiche all'avanguardia e in modo ottimizzato.

La comunicazione tra le due parti avviene per mezzo delle REST-API.



tailwindcss



VITE



Adattamenti implementativi

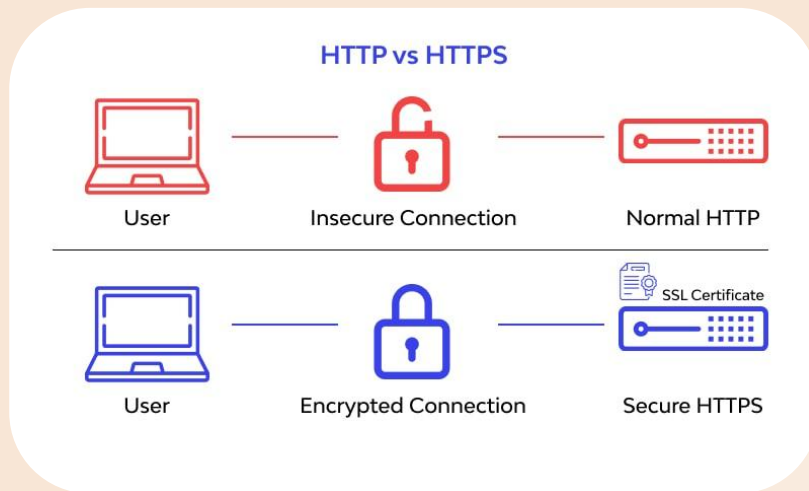
In fase di implementazione, è stato necessario operare alcune modifiche rispetto a quanto stabilito in progettazione:

- Trasferimento di alcuni metodi dai model ai controller (e.g. `calcolaTotaleImporti` in `GestionePromemoriaController`, `calcolaSaldoMancante` in `GestioneSaldoController`), per via della necessità di interazione con altre entità del sistema (e.g. `ConversioneImportoController`).
- Rimozione di alcuni metodi dai controller (e.g. `ordinaPerCategoria` in `GestioneMetodiPagamentoController`), rivelatisi inutili.
- Separazione della classe `MetodoPagamento` in due, con l'aggiunta dell'enum `TipoMetodoPagamento`. Scelta necessaria dovuta al fatto che in C# non è possibile definire attributi nelle enumerazioni.
- Modifica della visibilità di alcuni metodi, in modo da rispettare maggiormente l'incapsulamento (e.g. `generaEstrattoContoPrecedente` e `generaEstrattoContoAttuale` in `GestioneResocontiController`).



TLS e HTTPS

ASP.NET Core supporta nativamente TLS e HTTPS, semplificando la protezione delle comunicazioni tra client e server. Durante lo sviluppo, è possibile utilizzare certificati self-signed generati automaticamente dall'ambiente .NET. Il front-end, sviluppato con React, Tailwind e Vite può interagire con il back-end attraverso HTTPS configurando correttamente l'URL delle API.



Database

Come database relazionale è stato scelto Microsoft SQL Server, il quale offre una combinazione di affidabilità, performance e integrazione nativa con l'ecosistema .NET, su cui si basa il back-end della nostra applicazione.

Cashmonkey interagisce con Microsoft SQL Server tramite la metodologia di forza bruta in modo tale da avere un controllo diretto sulla definizione della struttura dei dati e sulle operazioni svolte sul database.

In questo modo è possibile anche ottimizzare ogni singola query per garantire prestazioni più elevate e rispettare il requisito non funzionale sulla velocità di memorizzazione e ricerca dei dati.



Prototipo

Si riportano alcune schermate di utilizzo del prototipo dell'applicativo (login e gestione dei movimenti).



Login

Accedi al tuo account CashMonkey

Username

 admin

Password

Login

Non hai un account? [Registrati ora](#)

© 2025 CashMonkey. Tutti i diritti riservati.

CashMonkey

Movimenti

▼ Filtri

+ Aggiungi

Movimenti

Metodi pagamento

Saldo

Obiettivo economico

Promemoria

Resoconti

Entrate

9.167,00 €

Uscite

410,50 €

Bilancio

\$ 8.756,50 €

Tipo	Data	Categoria	Metodo	Descrizione	Importo	Azioni
↗	07/05/2025	Svago	Postepay	Cena fuori	31,00 €	—
↘	07/05/2025	Svago	Revolut	Cena fuori	62,00 €	—
↗	06/05/2025	Casa	Salvadanaio	Affitto maggio	1.280,00 €	—
↗	05/05/2025	Salute	Bonifici SEPA	Rimborso spese	25,00 €	—
↘	03/05/2025	Svago	Revolut	Acquisto online	234,00 €	—
↘	03/05/2025	Casa	Postepay	Spesa	34,50 €	—
↗	03/05/2025	Finanza	Bonifici SEPA	Rimborso spese	550,00 €	—